ASSIGNMENT-10.2

HT NO:2303A51360

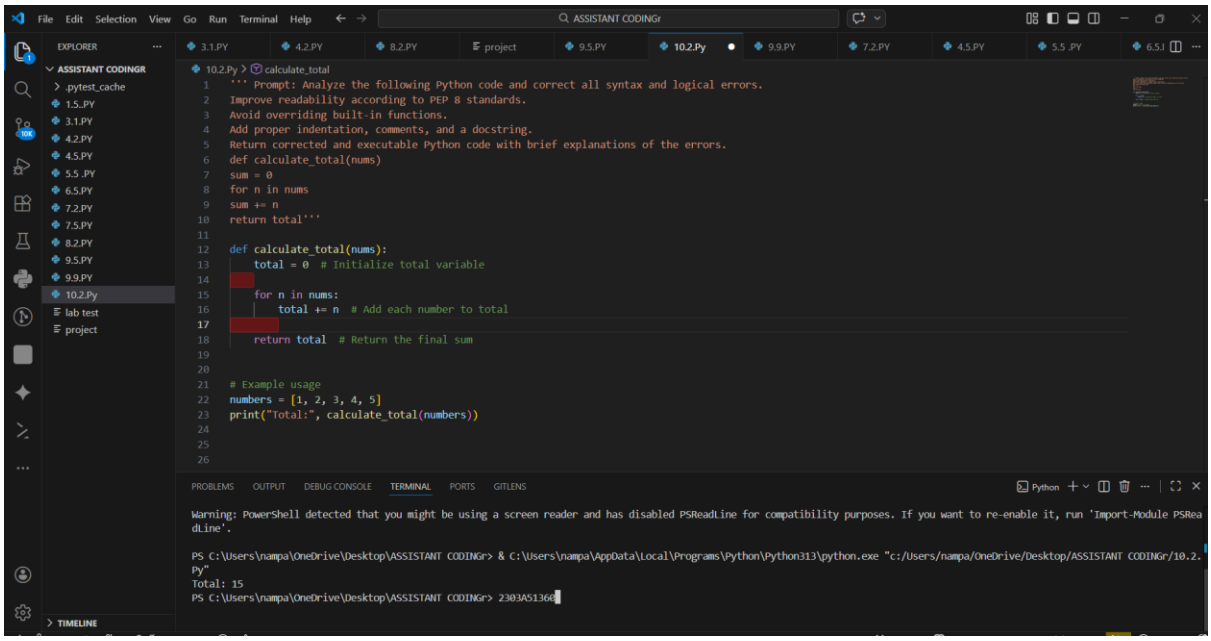
BATCH NO: 29

TASK-1:

PROMPT:

Analyze the following Python code, detect and correct all syntax and logical errors, and return executable code with brief explanations of the identified issues.

CODE:



OBSERVATION:

The original code had missing colons (:) after the function definition and for loop.

Indentation was incorrect, which causes syntax errors in Python.

The variable total was returned without being defined.

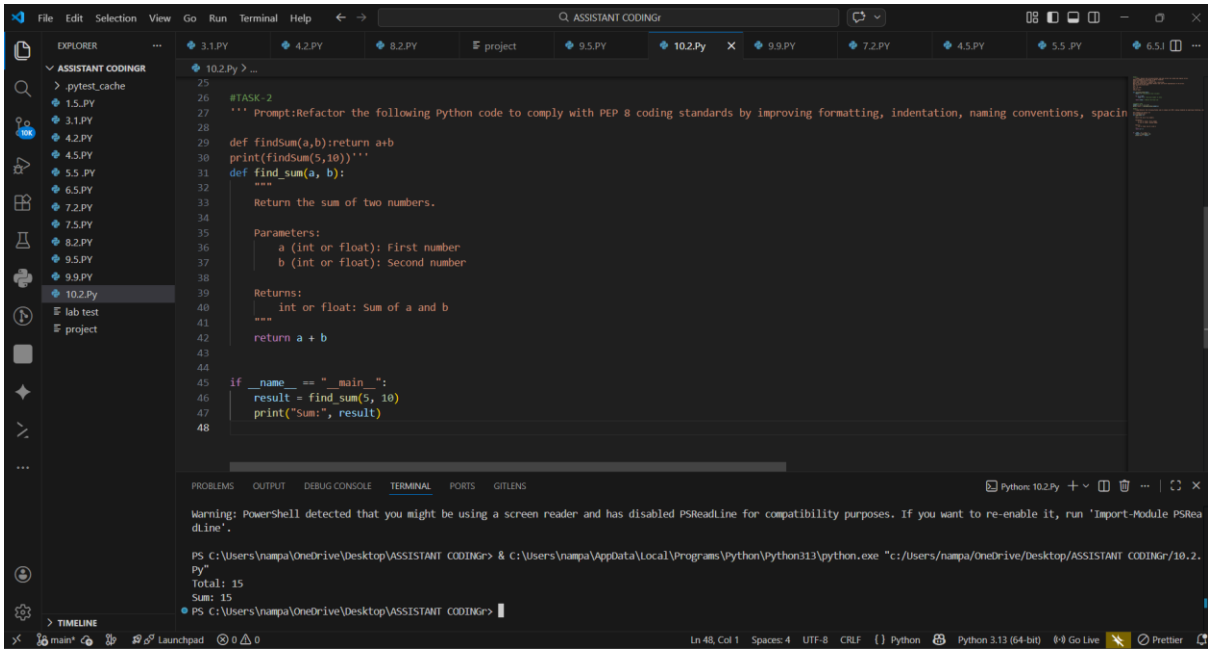
The variable name sum overrides Python’s built-in sum() function.

After correcting these issues, the code executes properly and follows good coding practices.

TASK-2:

PROMPT: Refactor the following Python code to comply with PEP 8 coding standards by improving formatting, indentation, naming conventions, spacing, and documentation; return well-structured and consistently formatted executable code.

CODE:



OBSERVATION:

1. The function name was changed from findSum to find_sum to follow snake_case naming convention.

2. Proper spacing was added after commas and around operators.

3. The one-line function was expanded into a properly indented multi-line format.

4. A docstring was added to improve readability and documentation.

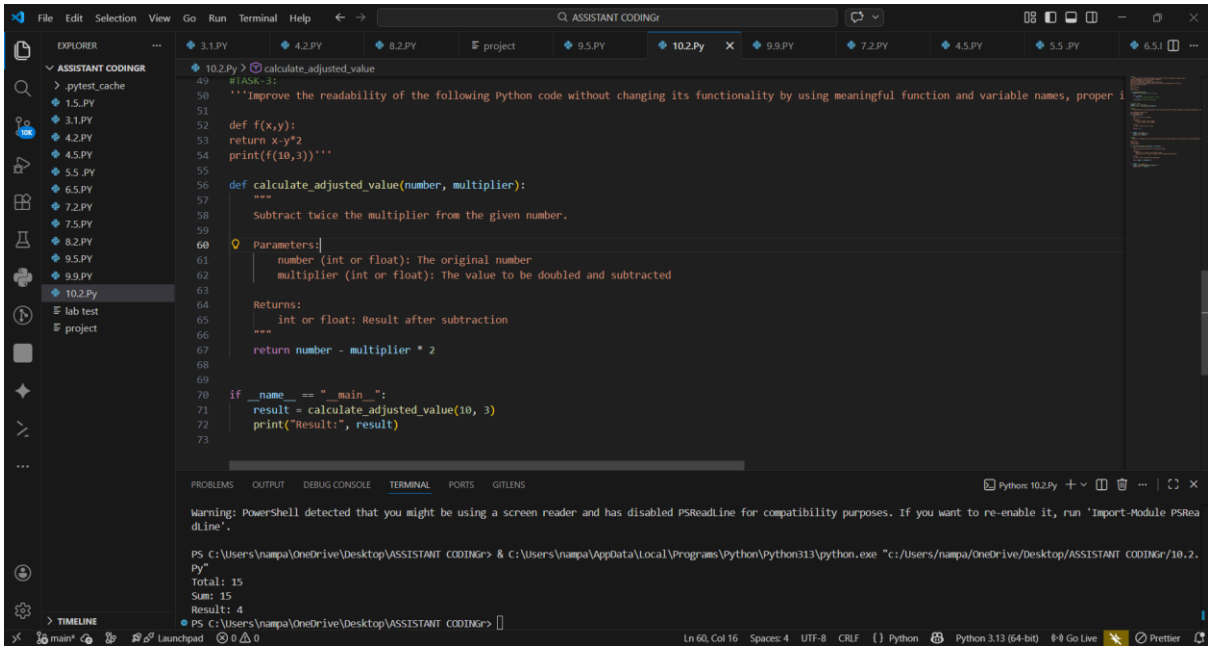
5. The if __name__ == "__main__": block was added for better program structure and maintainability.

TASK-3:

PROMPT:

Refactor the given Python code to improve readability and clarity without changing its functionality by using meaningful names, proper indentation, correct spacing, and a clear structure, and return the improved version.

CODE:



OBSERVATION:

1. The function name was changed to a meaningful name for better understanding.
2. Variable names were improved to clearly describe their purpose.
3. Proper indentation and spacing were applied for better readability.
4. A structured format was used to make the code more organized.

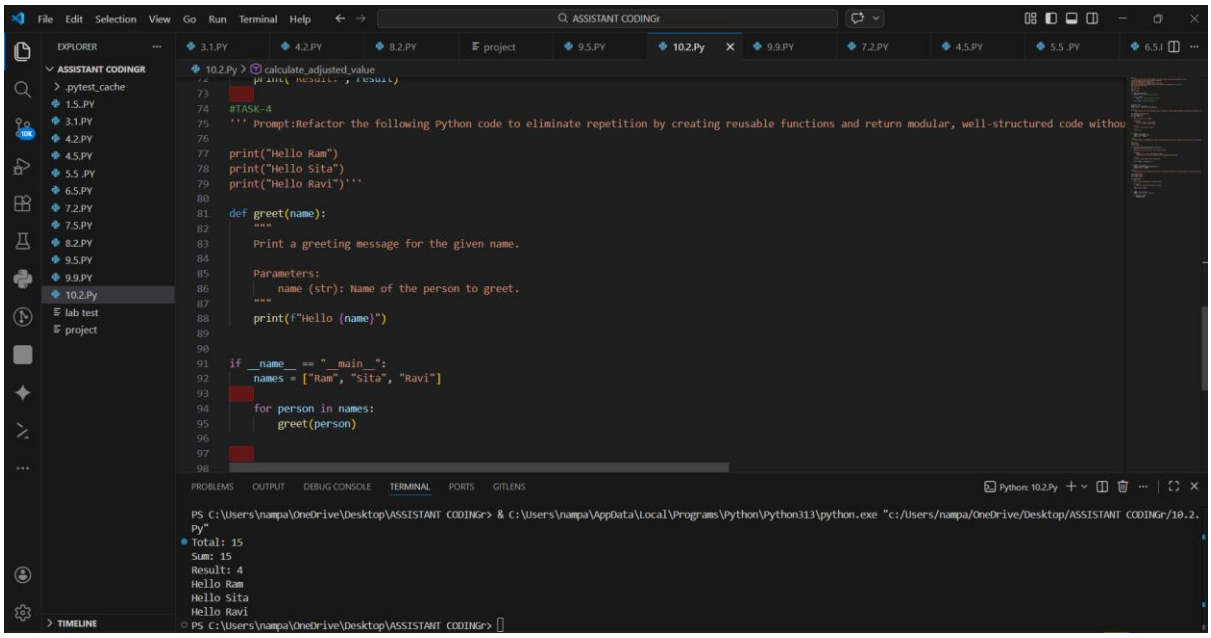
5. The functionality of the program remains unchanged while clarity is improved.

TASK-4:

PROMPT:

Refactor the following Python code to eliminate repetition by creating reusable functions and return modular, well-structured code without changing the program's output.

CODE:



OBSEVATION:

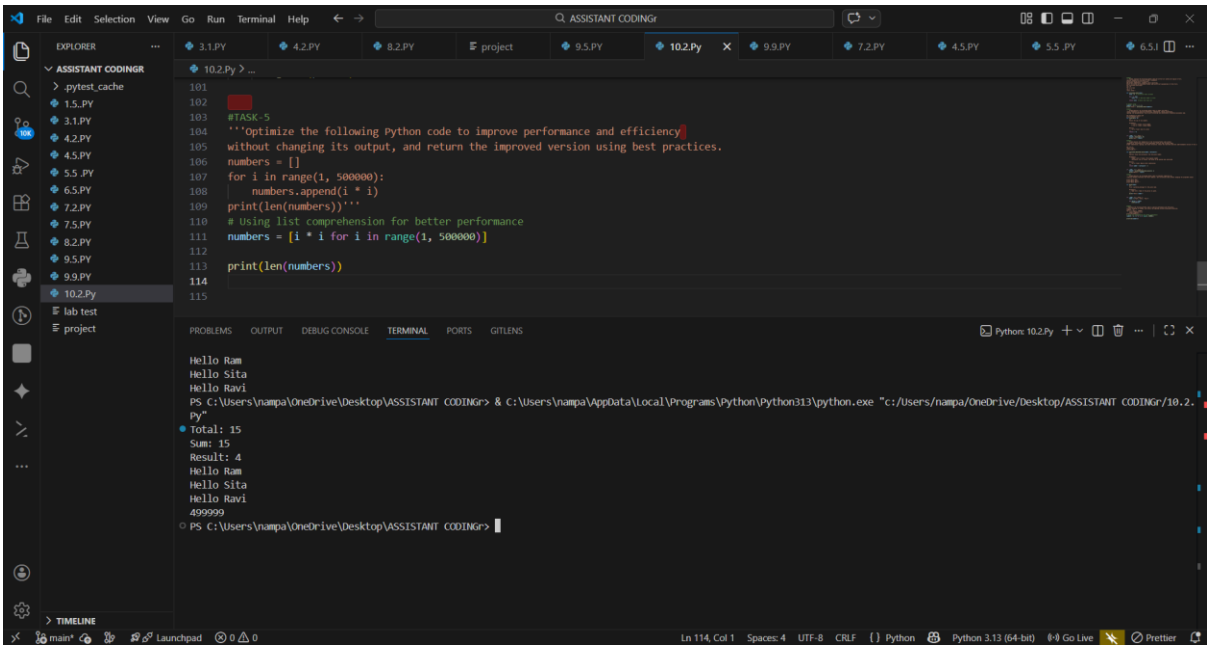
1. The repetitive print statements were replaced with a reusable function.
2. A greet() function was created to improve modularity.
3. A list and loop were used to avoid code duplication.
4. The structure is cleaner and easier to maintain or extend.
5. The program output remains unchanged while improving code quality.

TASK-5:

PROMPT:

Optimize the following Python code to improve performance and efficiency without changing its output, and return the improved version using best practices.

CODE:



OBSERVATION:

1. The for loop with append() was replaced by list comprehension.

2. List comprehension is faster and more Pythonic.

3. The code is now shorter and easier to read.

4. Performance is improved due to optimized internal implementation.

5. The output remains the same while efficiency is enhanced.